



A.D. 1308
unipg

DIPARTIMENTO
DI INGEGNERIA

Progetto Finale di
Signal Processing and Optimization for Big Data
Corso di Laurea in Ingegneria Informatica ed Elettronica
DIPARTIMENTO DI INGEGNERIA
A.A. 2024-2025

docente
Prof. Paolo BANELLI

Matrix Completion per MovieLens

Implementazione, analisi e confronto di algoritmi di Matrix Completion applicati al dataset
MovieLens

studente

355308 **Alice Fortuni** alice.fortuni@studenti.unipg.it

0. Indice

1	Introduzione	2
1.0.1	Matrix Completion e sistemi di raccomandazione	2
1.0.2	Dataset utilizzato	4
2	Approcci al problema di Matrix Completion	5
2.1	Minimizzazione del rango	5
2.2	Matrix Factorization	8
2.2.1	Feedback impliciti	11
3	Implementazione	13
3.1	Preparazione Dataset	13
3.2	Applicazione algoritmi	14
4	Risultati sperimentali	15
5	Conclusioni	22
6	Bibliografia	23

1. Introduzione

L'obiettivo di questo progetto è quello di affrontare il problema del completamento di matrici (*Matrix Completion*) nel contesto specifico dei sistemi di raccomandazione (*Recommender System*). Più in particolare, verranno analizzate e confrontate le prestazioni di diverse tecniche, applicate a un dataset reale, *MovieLens*, che è ampiamente utilizzato nella letteratura sui sistemi di raccomandazione.

Dopo un'introduzione al problema generale e al suo ruolo nei sistemi di raccomandazione, verranno descritti gli algoritmi principali utilizzati per la risoluzione del problema. Infine, sarà trattata l'implementazione e discussi i risultati degli esperimenti condotti.

1.0.1 Matrix Completion e sistemi di raccomandazione

Il problema di Matrix Completion è centrale in numerosi ambiti applicativi, dove i dati disponibili risultano parziali o incompleti a causa di vari limiti pratici, come la scarsità delle informazioni o i costi elevati associati all'acquisizione degli stessi. Questo problema è particolarmente rilevante nei sistemi di raccomandazione, ovvero sistemi progettati per fornire suggerimenti personalizzati che migliorano l'esperienza degli utenti e riducono il tempo necessario per la ricerca di contenuti pertinenti.

Un Recommender System analizza il comportamento passato degli utenti sfruttando le interazioni tra quest'ultimi e i diversi articoli. Tali interazioni vengono raccolte in matrici (*Interaction Matrix*, 1.1), in cui:

- le righe corrispondono agli utenti;

- le colonne rappresentano gli articoli;
- i valori delle celle indicano il feedback dell'utente, che può essere di tipo esplicito (come valutazioni numeriche) o implicito (visualizzazioni o altre interazioni osservabili).

	Item A	Item B	Item C	Item D	Item E
User 1	5				
User 2			4		
User 3					1
User 4		3			
User 5				2	

Figura 1.1: Esempio di Interaction Matrix.

Dato che non tutti gli utenti interagiscono con tutti gli articoli, queste matrici contengono numerosi valori mancanti. La stima di tali valori è fondamentale per migliorare la qualità delle raccomandazioni e offrire all'utente un'esperienza personalizzata ottimale [1]. Il completamento di matrici affronta proprio questo problema: prevedere i valori mancanti di una matrice parzialmente osservata.

Il problema di Matrix Completion può essere formalizzato come segue: data una matrice parzialmente osservata $M \in R^{m \times n}$ in cui M_{ij} rappresenta un valore noto per ogni $(i, j) \in \Omega$, con $\Omega \subseteq \{1, \dots, m\} \times \{1, \dots, n\}$ che identifica le posizioni degli elementi osservati, l'obiettivo è stimare i valori mancanti della matrice, sfruttando le informazioni disponibili.

Una delle principali assunzioni in questo contesto è che la matrice M abbia un rango basso o approssimativamente basso. Ciò implica che M può essere descritta mediante un numero limitato di fattori latenti, che catturano le relazioni essenziali tra righe e colonne. Ad esempio, una matrice di valutazioni di film può essere modellata come una combinazione di poche caratteristiche latenti, come il genere di un film, il cast o il regista. Questa struttura a rango basso permette di ricostruire valori mancanti attraverso tecniche di Matrix Completion, anche a fronte di una matrice molto sparsa, migliorando significativamente l'accuratezza e la personalizzazione delle raccomandazioni.

1.0.2 Dataset utilizzato

Il dataset utilizzato in questo progetto è una versione ulteriormente ridotta del dataset MovieLens Small (*ml-latest-small*), accessibile al seguente link. MovieLens è uno dei dataset più utilizzati nella ricerca sui sistemi di raccomandazione grazie alla sua semplicità e disponibilità di dati reali, rendendolo un punto di riferimento per lo sviluppo e il confronto di algoritmi in tale ambito.

La versione originale del dataset contiene 100836 valutazioni, raccolte da 610 utenti su 9724 film tra il 29 marzo 1996 e il 24 settembre 2018. Ogni valutazione è espressa su una scala da 0.5 a 5, con incrementi di 0.5 e rappresenta il livello di gradimento dell'utente verso un determinato film. Oltre alle valutazioni, il dataset contiene informazioni aggiuntive sui film e sugli utenti, che tuttavia non sono state utilizzate in questo progetto.

Per questo lavoro è stato selezionato un sottoinsieme di utenti e film in modo da ridurre la dimensione della matrice di interazione e ottimizzare i tempi di esecuzione degli algoritmi implementati.

L'obiettivo dell'utilizzo di questo dataset è quello di valutare le prestazioni degli algoritmi di Matrix Completion in un contesto realistico, analizzando la loro capacità di prevedere i valori mancanti e migliorare la qualità delle raccomandazioni.

2. Approcci al problema di Matrix Completion

In questa sezione vengono descritti i principali algoritmi per il completamento di matrici, che possono essere suddivisi in due famiglie:

- algoritmi basati sulla minimizzazione del rango della matrice;
- algoritmi basati sulla fattorizzazione della matrice (*Matrix Factorization*).

2.1 Minimizzazione del rango

La minimizzazione del rango rappresenta un approccio teorico fondamentale per il completamento di matrici, introdotto nello studio di Candès e Recht [2]. Questo metodo si basa sull'assunzione che la matrice originale abbia un rango basso o approssimativamente basso, una proprietà comune in molte applicazioni pratiche, come nei sistemi di raccomandazione.

In un contesto privo di rumore, il problema può essere formulato come segue:

$$\begin{aligned} \min_M \text{rank}(M) \\ \text{s.t. } M_\Omega = X_\Omega \end{aligned} \tag{1}$$

in cui l'obiettivo è ricostruire una matrice M minimizzandone il rango e vincolandola a coincidere, nelle posizioni specificate dall'insieme Ω , con i valori osservati della matrice incompleta X . In presenza di rumore, la formulazione si estende

includendo un termine di tolleranza sull'errore:

$$\begin{aligned} & \min_M \text{rank}(M) \\ \text{s.t. } & \|M_\Omega - X_\Omega\|_F \leq \delta \end{aligned} \quad (2)$$

dove $\|\cdot\|_F$ è la norma di Frobenius e $\delta > 0$ è un iperparametro di tolleranza al rumore.

Tuttavia, la minimizzazione diretta del rango è un problema *NP-hard*, rendendo le formulazioni (1) e (2) impraticabili per matrici di grandi dimensioni.

Per affrontare questa complessità e rendere il problema trattabile, il rango può essere sostituito con il suo rilassamento convesso, ovvero la norma nucleare, definita come:

$$\|M\|_* = \sum_{i=1}^{\min(m,n)} \sigma_i(M), \quad (3)$$

dove $\sigma_i(\mathbf{M})$ sono i valori singolari della matrice M .

Di fatto, minimizzare la norma nucleare tende a ridurre il numero di valori singolari significativi, favorendo implicitamente una soluzione a rango basso.

Questo rilassamento consente di riformulare il problema (1) come un problema di ottimizzazione convessa, facilmente risolvibile utilizzando strumenti di ottimizzazione standard come **CVX**, un package disponibile in **MATLAB**.

La formulazione del problema rilassato è:

$$\begin{aligned} & \min_{\mathbf{M}} \|M\|_* \\ \text{s.t. } & M_\Omega = X_\Omega \end{aligned} \quad (4)$$

In [4] è stato dimostrato che, sotto la condizione di forte incoerenza della matrice originale, è possibile ricostruire la matrice completa in modo esatto con alta probabilità, rendendo la minimizzazione della norma nucleare un approccio teoricamente solido ed efficace per il completamento di matrici.

Il problema della minimizzazione della norma nucleare può essere riformulato secondo un problema di programmazione semidefinita (*SDP*):

$$\begin{aligned} & \min_{M, W_1, W_2} \text{trace}(W_1) + \text{trace}(W_2) \\ \text{s.t. } & M_\Omega = X_\Omega \\ & \begin{bmatrix} W_1 & M \\ M^T & W_2 \end{bmatrix} \succeq 0 \end{aligned} \quad (5)$$

in cui si minimizza la traccia di due matrici semidefinite positive W_1 e W_2 vincolando il blocco matriciale

$$\begin{bmatrix} W_1 & M \\ M^T & W_2 \end{bmatrix}$$

ad essere semidefinito positivo, oltre che imporre l'uguaglianza dei valori nella matrice M con quelli delle entry osservate. Tuttavia, l'approccio è computazionalmente costoso, con complessità dell'ordine di $O(p(m+n)^3 + p^2(m+n) + p^3)$, dove p è il numero di osservazioni della matrice incompleta, rendendo il metodo impraticabile per matrici di grandi dimensioni ([3]).

Singular Value Thresholding (SVT)

Il Singular Value Thresholding è un algoritmo alternativo proposto per risolvere il problema di minimizzazione della norma nucleare. Invece di affrontare direttamente il problema espresso in (4), SVT utilizza una formulazione regolarizzata che combina la norma nucleare e la norma di Frobenius:

$$\begin{aligned} \min_{\mathbf{M}} \quad & \tau \|M\|_* + \frac{1}{2} \|M\|_F^2 \\ \text{s.t.} \quad & M_\Omega = X_\Omega \end{aligned} \tag{6}$$

dove:

- $\tau \geq 0$ è un iperparametro che regola il compromesso tra la minimizzazione della norma nucleare (favorendo una soluzione a rango basso) e la norma di Frobenius (favorendo stabilità numerica). È stato dimostrato in [5] che, per valori sufficientemente alti di τ , la soluzione al problema converge a quella del problema originale di minimizzazione della norma nucleare.

L'algoritmo SVT prevede una procedura iterativa basata sull'applicazione del *soft-thresholding* ai valori singolari di una matrice ausiliaria Y . Ad ogni iterazione, la soluzione viene aggiornata come segue:

$$\begin{aligned} M_k &= D_\tau(Y_{k-1}) \\ Y_k &= Y_{k-1} + \delta P_\Omega(X - M_k) \end{aligned} \tag{7}$$

dove:

- $D_\tau(Y)$ è l'operatore di soft-thresholding rispetto ai valori singolari di Y . Nello specifico, data la decomposizione SVD (*Singular Value Decomposition*) di Y :

$$Y = U \Sigma V^T, \quad \Sigma = \text{diag}(\{\sigma_i\}_{i \in [1, r]})$$

l'operatore D_τ è definito come:

$$D_\tau(Y) = U \text{diag}(\{(\sigma_i - \tau)^+\}_{i \in [1, r]}) V^T \quad (8)$$

dove:

$$(\sigma_i - \tau)^+ = \begin{cases} \sigma_i - \tau, & \text{se } \sigma_i - \tau > 0 \\ 0, & \text{altrimenti} \end{cases}$$

Quindi, l'operatore effettua una riduzione di ogni valore singolare σ_i di una quantità τ e pone a zero quelli che diventano negativi, favorendo soluzioni a rango basso.

- P_Ω è l'operatore di proiezione nell'insieme Ω , definito come:

$$[P_\Omega(A)]_{ij} = \begin{cases} A_{ij}, & \text{se } (i, j) \in \Omega \\ 0, & \text{altrimenti} \end{cases} \quad (9)$$

- δ rappresenta il passo di aggiornamento, tipicamente fissato in modo euristico a:

$$\delta = \frac{1.2 \cdot m \cdot n}{|\Omega|}$$

2.2 Matrix Factorization

La Matrix Factorization è un metodo alternativo agli approcci basati sulla minimizzazione della norma nucleare per risolvere problemi di completamento di matrici. Tale approccio evita l'utilizzo diretto della decomposizione a valori singolari, che risulta computazionalmente onerosa e poco scalabile per dataset di grandi dimensioni.

Il metodo è ampiamente utilizzato nei sistemi di raccomandazione e si basa sull'ipotesi che la matrice di interazione utente-articolo $M \in R^{m \times n}$ possa essere approssimata come il prodotto di due matrici a rango basso A e B :

$$M \approx AB^T$$

Nello specifico:

- $A \in R^{m \times r}$ (**User Matrix**): rappresenta i fattori latenti degli utenti, dove:
 - m è il numero di utenti e r è il numero di fattori latenti;
 - ogni riga $\mathbf{a}_i \in R^{1 \times r}$ corrisponde a un utente i ed è un vettore che cattura le sue preferenze implicite, come ad esempio il genere di film preferito o l'interesse per attori/registi specifici. Questi vettori non sono direttamente osservabili, ma vengono appresi durante la fase di ottimizzazione;
- $B \in R^{n \times r}$ (**Item Matrix**): rappresenta i fattori latenti degli articoli, dove:
 - n è il numero di articoli e r è il numero di fattori latenti;
 - ogni riga $\mathbf{b}_j \in R^{1 \times r}$ corrisponde ad un articolo j ed è un vettore che descrive le sue caratteristiche latenti. Analogamente a quelli degli utenti, anche questi fattori vengono appresi durante l'ottimizzazione.

Il rating previsto per l'utente i rispetto all'articolo j , indicato con M_{ij} , viene calcolato come il prodotto scalare tra i rispettivi vettori latenti:

$$M_{ij} \approx \mathbf{a}_i \cdot \mathbf{b}_j^T$$

L'obiettivo della Matrix Factorization è trovare le matrici A e B tali che il prodotto AB^T approssimi i valori osservati in X e permetta di predire i valori mancanti. Ciò viene formalizzato nel seguente problema di ottimizzazione:

$$\min_{A,B} \|(AB^T - X)_\Omega\|_F^2 + \lambda(\|A\|_F^2 + \|B\|_F^2) \quad (10)$$

dove:

- il primo termine è l'errore quadratico della stima rispetto ai valori osservati (Ω) in X ;
- il secondo termine è una regolarizzazione, utilizzata per evitare overfitting delle osservazioni note.

Questo problema può essere risolto iterativamente utilizzando algoritmi come *Gradient Descent (GD)* o *Alternating Least Squares (ALS)*.

Gradient Descent (GD)

Il metodo di Gradient Descent ottimizza le matrici A e B iterativamente, cercando di ridurre l'errore tra la matrice stimata e i valori osservati in X . Nello specifico, ad ogni iterazione, i valori nella *User Matrix* e nella *Item Matrix* vengono aggiornati seguendo la direzione opposta al gradiente della funzione obiettivo:

$$L(A, B) = \sum_{(i,j) \in \Omega} (\mathbf{a}_i \cdot \mathbf{b}_j^T - x_{ij})^2 + \lambda(\|\mathbf{a}_i\|^2 + \|\mathbf{b}_j\|^2) \quad (11)$$

Gli aggiornamenti per i singoli valori a_{ik} e b_{jk} sono dati da:

$$\begin{aligned} a_{ik}^{(t)} &= a_{ik}^{(t-1)} - \mu \frac{\partial L(\cdot)}{\partial a_{ik}} \\ b_{jk}^{(t)} &= b_{jk}^{(t-1)} - \mu \frac{\partial L(\cdot)}{\partial b_{jk}} \end{aligned}$$

dove μ è il *learning rate*.

Calcolando esplicitamente i gradienti otteniamo:

$$a_{ik}^{(t)} = a_{ik}^{(t-1)} - 2\mu(e_{ij}^{(t-1)}b_{jk}^{(t-1)} + \lambda a_{ik}^{(t-1)}) \quad (12)$$

$$b_{jk}^{(t)} = b_{jk}^{(t-1)} - 2\mu(e_{ij}^{(t-1)}a_{ik}^{(t-1)} + \lambda b_{jk}^{(t-1)}) \quad (13)$$

dove e_{ij} è l'errore relativo per l'elemento (i, j) e corrisponde alla differenza fra il valore stimato e quello osservato:

$$e_{ij} = \mathbf{a}_i \cdot \mathbf{b}_j^T - x_{ij}$$

Alternating Least Squares (ALS)

L'algoritmo di Alternating Least Squares affronta il problema 10 suddividendolo in due sotto-problemi più semplici che vengono risolti alternativamente mantenendo una delle due matrici fissata.

- Per un dato utente i , il vettore $\mathbf{a}_i$ della matrice A viene aggiornato minimizzando:

$$L(\mathbf{a}_i) = \sum_{j \in \Omega_i} (x_{ij} - \mathbf{a}_i \cdot \mathbf{b}_j^T)^2 + \lambda \|\mathbf{a}_i\|^2 \quad (14)$$

dove Ω_i rappresenta l'insieme degli indici degli item valutati dall'utente i .

Dall'equazione 14, imponendo il gradiente di $L(\mathbf{a}_i)$ rispetto ad $\mathbf{a}_i$ nullo, otteniamo l'aggiornamento:

$$\mathbf{a}_i = \sum_{j \in \Omega_i} x_{ij} \mathbf{b}_j \left(\sum_{j \in \Omega_i} \mathbf{b}_j^T \mathbf{b}_j + \lambda I \right)^{-1}$$

che può essere scritto in forma compatta, usando la matrice $B_{\Omega_i} \in R^{|\Omega_i| \times r}$, contenente solo le righe di B corrispondenti agli item osservati da i :

$$\mathbf{a}_i = \mathbf{x}_i B_{\Omega_i} (B_{\Omega_i}^T B_{\Omega_i} + \lambda I)^{-1} \quad (15)$$

dove $\mathbf{x}_i$ è il vettore dei valori osservati per l'utente i .

- Analogamente, per un dato item j , il vettore $\mathbf{b}_j$ della matrice B viene aggiornato attraverso:

$$\mathbf{b}_j = \mathbf{x}_j A_{\Omega_j} (A_{\Omega_j}^T A_{\Omega_j} + \lambda I)^{-1} \quad (16)$$

dove $\mathbf{x}_j$ è il vettore dei valori osservati per l'item j .

2.2.1 Feedback impliciti

Gli algoritmi di Matrix Factorization visti, che risolvono il problema in 10, vengono utilizzati in caso di feedback espliciti, ovvero quando gli utenti forniscono una valutazione diretta rispetto agli item. Tuttavia, in molti scenari, tali feedback non sono disponibili o i dati raccolti vengono interpretati come feedback impliciti. In questo contesto, i valori nella matrice di interazione indicano il livello di confidenza in una relazione positiva tra utente e item: valori alti suggeriscono una probabile preferenza, mentre i valori nulli indicano solo una mancanza di informazione, non una disapprovazione.

Per adattare la tecnica di Matrix Factorization al contesto dei feedback impliciti, in [6] sono state introdotte:

- Φ : una matrice binaria che rappresenta la preferenza implicita degli utenti rispetto agli item, in cui ogni elemento (i, j) viene derivato dalla matrice X come segue:

$$\phi_{ij} = \begin{cases} 1 & \text{se } x_{ij} > 0 \\ 0 & \text{se } x_{ij} = 0 \end{cases}$$

Un valore $\phi_{ij} = 1$ indica che l'utente i ha interagito con l'item j , mentre $\phi_{ij} = 0$ riflette un'assenza di informazione.

- C : una matrice di confidenza che assegna un peso a ciascun elemento di Φ . Gli elementi c_{ij} sono definiti come:

$$c_{ij} = 1 + \alpha x_{ij},$$

dove $\alpha > 0$ è una costante empiricamente fissata a $\alpha = 40$.

Nello specifico, avremo che:

- $c_{ij} = 1$ per $x_{ij} = 0$, indicando una bassa confidenza;
- c_{ij} cresce all'aumentare di x_{ij} , suggerendo che interazioni più frequenti implicano una maggiore confidenza della preferenza implicita.

In questo ambito, il problema di ottimizzazione da risolvere è il seguente:

$$\min_{A,B} \sum_{i,j} c_{ij} (\phi_{ij} - \mathbf{a}_i \cdot \mathbf{b}_j^T)^2 + \lambda (\|\mathbf{a}_i\|^2 + \|\mathbf{b}_j\|^2) \quad (17)$$

Analogamente al caso dei feedback espliciti, il problema può essere risolto utilizzando i metodi iterativi di *Gradient Descent (GD)* o di *Alternating Least Squares (ALS)*. In questo lavoro è stato trattato solo il caso di Gradient Descent.

Gradient Descent per feedback impliciti

Come visto per i feedback espliciti, il metodo di Gradient Descent aggiorna iterativamente le matrici A e B minimizzando la funzione obiettivo 17. In questo caso, gli aggiornamenti per i singoli valori a_{ik} e b_{jk} sono:

$$a_{ik}^{(t)} = a_{ik}^{(t-1)} + 2\mu \left(c_{ij} (\phi_{ij} - \mathbf{a}_i \cdot \mathbf{b}_j^T) b_{jk}^{(t-1)} - \lambda a_{ik}^{(t-1)} \right) \quad (18)$$

$$b_{jk}^{(t)} = b_{jk}^{(t-1)} + 2\mu \left(c_{ij} (\phi_{ij} - \mathbf{a}_i \cdot \mathbf{b}_j^T) a_{ik}^{(t-1)} - \lambda b_{jk}^{(t-1)} \right) \quad (19)$$

3. Implementazione

L'intero progetto è stato sviluppato in ambiente MATLAB, adatto per affrontare il problema di Matrix Completion, data la sua efficienza nel gestire calcoli complessi e operazioni su matrici sparse.

Il punto d'ingresso al progetto è il file `main.m`, che ne rappresenta il corpo centrale contenendo tutte le parti essenziali, dalla preparazione del dataset all'esecuzione degli algoritmi e valutazione dei risultati.

3.1 Preparazione Dataset

Dopo l'importazione e una breve fase di esplorazione del dataset per comprenderne la struttura e le caratteristiche principali, è stata selezionata una porzione più gestibile di dati ai fini dell'analisi. In particolare:

- sono stati selezionati i primi 500 film;
- sono stati considerati gli utenti che hanno valutato almeno 5 film fra quelli selezionati.

Il dataset risultante comprende 478 utenti e 500 film, rappresentando un sottoinsieme significativo di quello originale, ma riducendone la complessità computazionale. Successivamente, il dataset ridotto è stato suddiviso in più coppie di training set e test set, variando la percentuale di valutazioni utilizzate per il test set, al fine di valutare il comportamento degli algoritmi in diverse condizioni di sparsità. In particolare, sono stati definiti tre scenari distinti in cui il test set conteneva rispettivamente il 10%, il 30% e il 70% delle valutazioni totali e il dataset di training la

parte complementare.

La suddivisione del dataset tra training e test è stata implementata utilizzando una funzione dedicata, `dataset_splitting`, che garantisce una distribuzione bilanciata dei dati e assicura che ogni utente abbia almeno una valutazione nel dataset di training.

3.2 Applicazione algoritmi

Dopo la preparazione del dataset, sono stati applicati alcuni degli algoritmi discussi in precedenza. Per ognuno è stata creata una funzione specifica:

- `nuclear_norm_minimization`: implementa la minimizzazione della norma nucleare (4) utilizzando il pacchetto `cvx` con la funzione `norm_nuc`;
- `svt`: implementa l'algoritmo iterativo di Singular Value Thresholding (7). Gli iperparametri, τ (soglia sui valori singolari) e δ (passo di aggiornamento), sono stati ottimizzati attraverso grid search, implementata nella funzione `svt_grid_search`;
- `mf_explicit_gd`: implementa l'algoritmo di Matrix Factorization per feedback espliciti attraverso Gradient Descent (2.2). I parametri ottimali, quali μ (*learning rate*), λ (termine di regolarizzazione) ed r (rango delle matrici A e B), sono stati scelti attraverso `gd_grid_search`;
- `mf_explicit_als`: implementa la Matrix Factorization per feedback espliciti utilizzando Alternating Least Squares (2.2). Gli iperparametri λ e r sono stati ottimizzati attraverso `als_grid_search`;
- `mf_implicit_gd`: implementa la Matrix Factorization per feedback impliciti utilizzando Gradient Descent (2.2.1). I parametri λ , μ , r e α (fattore di confidenza) sono stati ottimizzati con `gd_implicit_grid_search`.

Dato che le valutazioni nel dataset originale sono comprese tra 0.5 e 5 con incrementi di 0.5, è stata implementata la funzione `normalize_matrix` per riportare i risultati degli algoritmi, definiti su un range di valori più ampio, all'intervallo originale.

4. Risultati sperimentali

In questo capitolo vengono discussi i risultati sperimentali ottenuti dall'applicazione degli algoritmi.

Ogni algoritmo è stato valutato su tre diversi livelli di sparsità, simulati nascondendo progressivamente una percentuale crescente di valori nella matrice originale, come descritto nella sezione 3.1.

La valutazione della qualità delle matrici ricostruite è stata effettuata utilizzando il *Normalized Mean Squared Error (NMSE)*, espresso in decibel (dB) e definito come:

$$\text{NMSE (dB)} = 10 \cdot \log_{10} \left(\frac{\|X - M\|_F^2}{\|X\|_F^2} \right)$$

dove X rappresenta, nel caso in esame, la matrice di test, e M la matrice ricostruita. Il confronto è stato effettuato esclusivamente sulle celle in cui la matrice di test contiene valutazioni non nulle ($x_{ij} \neq 0$), questo perchè le celle nulle rappresentano dati mancanti o non osservati, e quindi non possono essere utilizzate per una valutazione significativa.

Oltre alla qualità della ricostruzione rispetto alla matrice di test, sono stati confrontati:

- tempi di esecuzione: al fine di valutare l'efficienza computazionale degli algoritmi;
- andamento dell'NMSE durante le iterazioni: calcolato sull'intera matrice originale, includendo sia i valori osservati che quelli nascosti, e la matrice ricostruita. In primis, questo ha permesso di analizzare come varia, durante le iterazioni, la qualità globale della ricostruzione. Inoltre, ha consentito di

ottenere informazioni sulla velocità di convergenza dell'algoritmo, valutando quanto rapidamente l'errore diminuisce nelle fasi iniziali, e di verificare la stabilità della soluzione, che si riflette in un valore costante dell'NMSE globale. Infine, l'analisi dell'NMSE può evidenziare eventuali fenomeni di overfitting, che si manifestano con un aumento dell'errore durante le iterazioni a causa di un adattamento eccessivo dell'algoritmo ai dati osservati, a scapito della generalizzazione sui dati nascosti.

In seguito vengono riportati e discussi i risultati ottenuti dagli algoritmi applicati.

Minimizzazione della norma nucleare

In Tabella 4.1 sono riportati i risultati dell'algoritmo di minimizzazione della norma nucleare rispetto ai diversi gradi di sparsità, ottenuti variando il *Test Ratio*:

Test Ratio	NMSE (dB)	Execution Time (s)
0.1	-11.111	239.6417
0.3	-10.8626	168.0753
0.7	-9.3536	30.0005

Tabella 4.1: Risultati della minimizzazione della norma nucleare.

Quando la percentuale di valori oscurati è più bassa (*Test Ratio* più basso), la matrice osservata di training contiene un numero maggiore di valori noti. Ciò implica che la minimizzazione della norma nucleare deve soddisfare un numero maggiore di vincoli, cercando una matrice a rango basso compatibile con una quantità più ampia di dati osservati. Questo rende il problema di ottimizzazione più complesso, che si traduce in un aumento del tempo di esecuzione (*Execution Time*), come notiamo nel caso di *Test Ratio* pari a 0.1.

Al contrario, all'aumentare della sparsità (*Test Ratio* più alto), la quantità di dati osservati diminuisce, semplificando il problema di ottimizzazione, con una conseguente riduzione del tempo di esecuzione, ma anche una qualità di ricostruzione inferiore, indicata dall'aumento del valore di NMSE.

In conclusione, come prevedibile, la minimizzazione della norma nucleare funziona meglio con più dati osservati, migliorando la qualità della ricostruzione, ma al costo di tempi di esecuzione più elevati.

SVT

In Figura 4.1 è mostrato il grafico che rappresenta l'andamento dell'NMSE globale durante le iterazioni per l'algoritmo SVT.

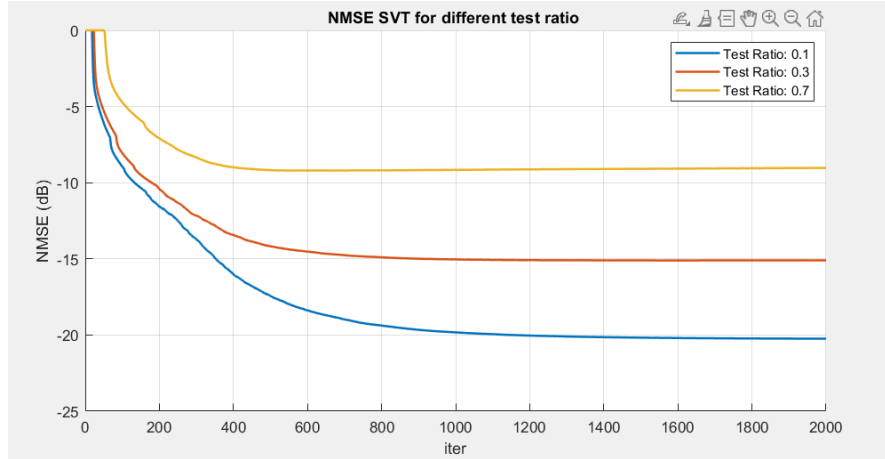


Figura 4.1: Andamento dell'NMSE (dB) per SVT durante le iterazioni.

Si nota, per tutti i Test Ratio, una progressiva diminuzione dell'NMSE, seguita da una stabilizzazione, che indica la capacità dell'algoritmo di convergere verso una soluzione che approssima sempre meglio la matrice originale. L'errore finale dipende dal Test Ratio: con un Test Ratio inferiore (10%), l'NMSE raggiunge valori più bassi, grazie alla maggiore disponibilità di dati osservati che guidano la ricostruzione. Al contrario, con un Test Ratio più alto (70%), l'NMSE si stabilizza su valori più elevati, evidenziando la difficoltà dell'algoritmo nel ricostruire la matrice quando le informazioni disponibili sono limitate.

I risultati sul dataset di test e il tempo di esecuzione per ogni livello di sparsità, sono riportati in Tabella 4.2, da cui emerge che l'errore finale (NMSE) sul dataset di test aumenta all'aumentare del Test Ratio, confermando una peggiore capacità di ricostruzione all'aumentare della sparsità.

Rispetto alla minimizzazione della norma nucleare, SVT fornisce risultati di qualità di ricostruzione inferiori, ma migliora i tempi di esecuzione.

Test Ratio	SVT NMSE (dB)	Execution Time (s)
0.1	-10.3938	48.6867
0.3	-10.0366	48.1734
0.7	-7.7836	49.0998

Tabella 4.2: Risultati della ricostruzione tramite SVT.

Matrix Factorization per feedback espliciti con GD e ALS

Nelle Figure 4.2 e 4.3 sono mostrati i grafici relativi all'andamento dell'errore di ricostruzione globale per gli algoritmi di Matrix Factorization basati su feedback espliciti. Si osserva in entrambi gli algoritmi, Gradient Descent e Alternating Least Squares, una diminuzione dell'NMSE durante le iterazioni, suggerendo una progressiva ottimizzazione della soluzione. Tuttavia, GD presenta un miglioramento più graduale e continua a ridurre l'NMSE, per i Test Ratio più bassi, anche oltre le 2000 iterazioni. ALS, invece, converge rapidamente, stabilizzandosi entro poche iterazioni.

Coerentemente con le aspettative, in entrambi i casi, la qualità della ricostruzione globale migliora con Test Ratio più bassi (minore sparsità), grazie alla maggiore quantità di dati osservati.

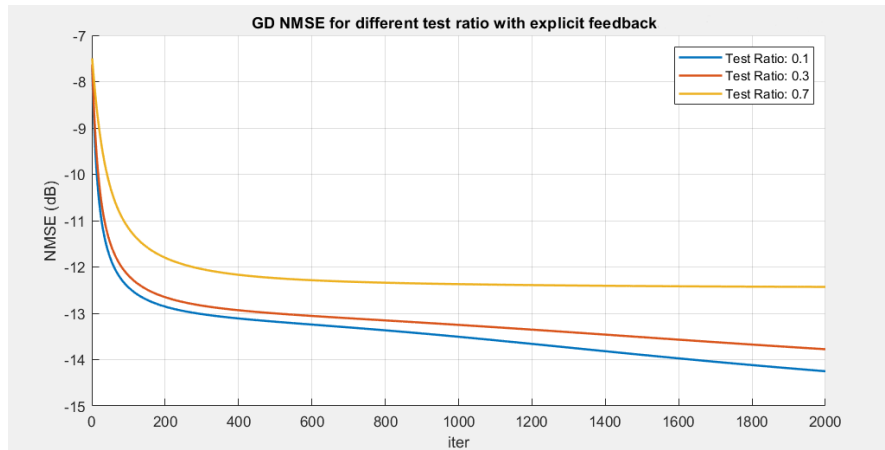


Figura 4.2: Andamento dell'NMSE (dB) per Matrix Factorization tramite GD durante le iterazioni.

Anche le Tabelle 4.3 e 4.4, che riportano i risultati di ricostruzione rispetto alla

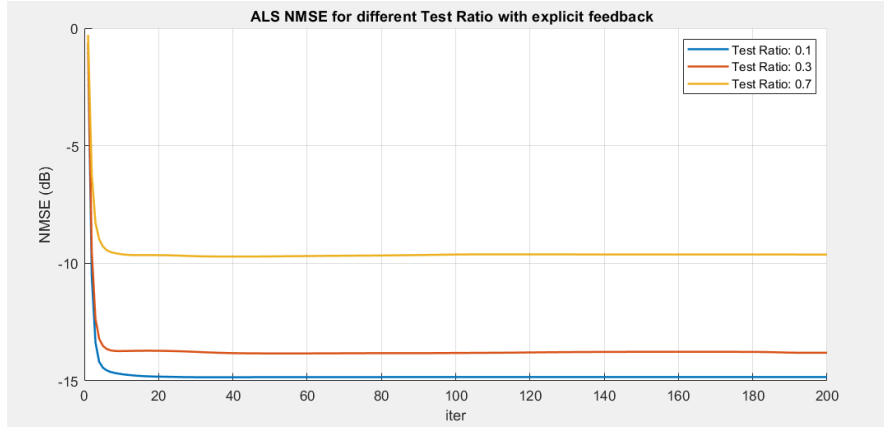


Figura 4.3: Andamento dell'NMSE (dB) per Matrix Factorization tramite ALS durante le iterazioni.

matrice di test e i tempi di esecuzione, confermano queste osservazioni. GD ottiene un NMSE leggermente migliore rispetto ad ALS per Test Ratio bassi (0.1 e 0.3), ma una qualità di ricostruzione decisamente superiore per Test Ratio elevati (0.7), evidenziando una maggiore robustezza in condizioni di elevata sparsità. Per quanto riguarda i tempi di esecuzione, ALS è nettamente più rapido in tutte le condizioni, con un tempo di calcolo inferiore di un ordine di grandezza rispetto a GD.

Il progressivo calo del tempo di esecuzione con l'aumento della sparsità è coerente con la riduzione del numero di dati osservati, che riduce la complessità computazionale per entrambi gli algoritmi.

Test Ratio	GD NMSE (dB)	Execution Time (s)
0.1	-12.0922	30.6424
0.3	-12.0558	23.4916
0.7	-11.4413	14.4091

Tabella 4.3: Risultati della ricostruzione tramite GD.

Complessivamente, i metodi basati sulla Matrix Factorization per feedback espliciti sono risultati migliori sia in termini di qualità della ricostruzione sia di efficienza computazionale rispetto agli approcci di minimizzazione della norma nucleare.

Test Ratio	ALS NMSE (dB)	Execution Time (s)
0.1	-11.9314	2.0670
0.3	-11.7566	1.2457
0.7	-10.1376	0.95323

Tabella 4.4: Risultati della ricostruzione tramite ALS.

Matrix Factorization per feedback impliciti con GD

In Figura 4.4 è mostrato l'andamento dell'NMSE globale nell'algoritmo di Matrix Factorization basato su feedback impliciti, implementato attraverso Gradient Descent.

Il grafico mostra che l'errore di ricostruzione globale, in tutte le condizioni di sparsità, diminuisce progressivamente durante le iterazioni, indicando che l'algoritmo ottimizza efficacemente la soluzione. Tuttavia, si osserva che l'NMSE raggiunge valori stazionari superiori rispetto agli altri metodi, indicando una minore capacità di ricostruzione accurata della matrice originale.

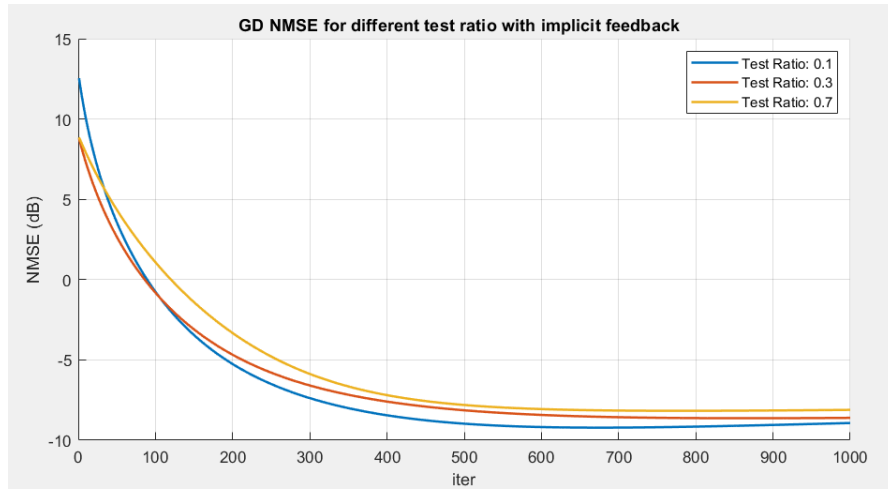


Figura 4.4: Andamento dell'NMSE(dB) per Matrix Factorization tramite GD durante le iterazioni, per feedback impliciti.

Come atteso, i valori riportati in Tabella 4.5 evidenziano che l'NMSE relativo alla matrice di test è più alto rispetto agli altri metodi. Anche in questo caso, all'aumentare della sparsità, l'errore di ricostruzione peggiora.

Test Ratio	GD NMSE (dB)	Execution Time (s)
0.1	-8.8773	135.6585
0.3	-8.7147	135.6701
0.7	-8.0227	145.6484

Tabella 4.5: Risultati della ricostruzione tramite GD per feedback impliciti.

I tempi di esecuzione risultano significativamente più elevati rispetto alle altre tecniche, attribuibile alla maggiore complessità computazionale dell'algoritmo. In particolare, nei metodi basati su feedback impliciti, l'ottimizzazione considera tutte le coppie (i, j) nella matrice di interazione, includendo sia le celle osservate che quelle non osservate ([6]), a differenza dei metodi per feedback espliciti, che lavorano unicamente sulle celle osservate.

I risultati, complessivamente peggiori rispetto agli altri metodi di MF, possono essere spiegati dalla natura del dataset in esame, MovieLens, che contiene valutazioni numeriche esplicite da parte degli utenti rispetto ai film.

Tabella Riassuntiva dei Risultati

Per fornire una panoramica completa delle performance degli algoritmi analizzati, sono riportate due tabelle riassuntive: la prima relativa ai tempi di esecuzione e la seconda all'errore di ricostruzione (NMSE in dB). Per agevolare il confronto, sono stati evidenziati i risultati migliori.

Test Ratio	Nuclear Norm (s)	SVT (s)	GD Explicit (s)	ALS (s)	GD Implicit (s)
0.1	239.6417	48.6867	30.6424	2.0670	135.6585
0.3	168.0753	48.1734	23.4916	1.2457	135.6701
0.7	30.0005	49.0998	14.4091	0.9532	145.6484

Tabella 4.6: Confronto dei tempi di esecuzione per i diversi algoritmi.

Test Ratio	Nuclear Norm (dB)	SVT (dB)	GD Explicit (dB)	ALS (dB)	GD Implicit (dB)
0.1	-11.111	-10.3938	-12.0922	-11.9314	-8.8773
0.3	-10.8626	-10.0366	-12.0558	-11.7566	-8.7147
0.7	-9.3536	-7.7836	-11.4413	-10.1376	-8.0227

Tabella 4.7: Confronto dell'NMSE per i diversi algoritmi.

5. Conclusioni

L'analisi comparativa degli algoritmi di Matrix Completion ha evidenziato che la tecnica di Matrix Factorization basata su feedback espliciti è risultata la più adatta al dataset MovieLens, grazie alla natura esplicita delle valutazioni. Nello specifico, ALS si è distinto per tempi di esecuzione ridotti (4.6), mentre GD ha mostrato una maggiore robustezza a fronte di scenari molto sparsi, garantendo una qualità di ricostruzione migliore (4.7).

I metodi per feedback impliciti e quelli basati sulla minimizzazione della norma nucleare hanno offerto risultati inferiori, rispettivamente a causa della loro mancata ottimizzazione per dati espliciti e della maggiore complessità computazionale.

6. Bibliografia

- [1] Z. Chen and S. Wang, “A review on matrix completion for recommender systems,” *Knowledge and Information Systems*, vol. 64, pp. 1–34, 2022. DOI: <https://doi.org/10.1007/s10115-021-01629-6>.
- [2] E. J. Candès and B. Recht, “Exact Matrix Completion via Convex Optimization,” *Foundations of Computational Mathematics*, vol. 9, pp. 717–772, 2009. DOI: <https://doi.org/10.1007/s10208-009-9045-5>.
- [3] X. P. Li, L. Huang, H. C. So, and B. Zhao, “A Survey on Matrix Completion: Perspective of Signal Processing,” *arXiv preprint*, arXiv:1901.10885, 2019. DOI: <https://doi.org/10.48550/arXiv.1901.10885>.
- [4] E. J. Candès and T. Tao, “The power of convex relaxation: Near-optimal matrix completion,” *arXiv preprint*, arXiv:0903.1476, 2009. [Online]. Available: <http://arxiv.org/abs/0903.1476>.
- [5] J. F. Cai, E. J. Candès, and Z. Shen, “A singular value thresholding algorithm for matrix completion,” *SIAM Journal on Optimization*, vol. 20, no. 4, pp. 1956–1982, Jan. 2010. DOI: <https://doi.org/10.1137/080738970>.
- [6] Y. Hu, Y. Koren, and C. Volinsky, “Collaborative filtering for implicit feedback datasets,” *2008 IEEE International Conference on Data Mining*, pp. 263–272, 2008. DOI: <https://doi.org/10.1109/ICDM.2008.22>.